

AltheronML Symposium OS

Product Requirement Document (PRD)

An Operating System for National Level Technical & Non-Technical Symposiums
Specifically Configured for Kuppam Engineering College
Department of Computer Science & Engineering
(Artificial Intelligence & Machine Learning)

Version: 2.1.0 (2k26 Edition)

Author: Symposium Technical Committee

Database: Firebase Real-time Firestore Sync

Status: Approved & Production Ready

Table of Contents

1. Executive Summary & Vision	Page 3
2. Product Scope & Functional Goals	Page 3
3. User Roles & Access Control Permission Matrix	Page 4
4. System Architecture & Component Specifications	Page 5
4.1. Secure Login & Bootstrapping Panel	Page 5
4.2. Public Self-Registration & Pass Issuance	Page 5
4.3. Registration Desk Console (Volunteers)	Page 6
4.4. Event Host & Evaluation Console	Page 6
4.5. Live Support Helpline Integration	Page 7
5. Database Architecture & Firestore Schemas	Page 7
6. Technical Stack, Security Rules & Non-Functional Requirements	Page 9
7. Offline Synchronization Architecture	Page 10

1. Executive Summary & Vision

The AltheronML Symposium Management System (dubbed "Symposium OS") is a high-performance full-stack web application designed to facilitate seamless, real-time coordination of academic symposiums at Kuppam Engineering College. Modern campus symposiums suffer from high latency, check-in logjams, manual scoring errors, and internet connectivity blackouts. Symposium OS resolves these challenges by providing an administrative engine that integrates offline-first local queue sync, real-time Firestore database listener push, and clean role-based dashboards.

This product requirement document outlines the definitive functional capabilities, data schemas, sync logic, and styling directives for the 2k26 edition of the software.

2. Product Scope & Functional Goals

The operational scope of the system covers four key execution phases:

1. Pre-Event Registration: Allowing online self-registration for individuals and teams, complete with QR code pass generation and print utility layouts.
2. Reception & Check-in Logistics: Empowering the registration team with real-time searches, check-in logs, and live payment status tracking.
3. Evaluation & Evaluation Logistics: Providing hosts and judges with customized scoring configurations, batch division widgets, and instant result publication hooks.
4. Live Support Coordination: Connecting volunteers and hosts to a central helpline support structure.

Operational Goals:

- * Zero Delay: Immediate cloud synchronization so that check-ins and registrations update instantly across all team devices.
- * Offline Fallback: Gracefully handling campus network dropouts by saving registrations and status changes locally, then writing them back once connection restores.
- * High-Density Roster Space: Maximize screen efficiency with collapsible menus so volunteers can manage heavy participant queues without layout crowding.

3. User Roles & Access Control Permission Matrix

To secure the system from unauthorized scoring modifications or check-ins, the system enforces a strict role-based permission hierarchy linked to authenticated accounts.

Role	Permitted Capabilities & Actions
Super Admin	Full read and write permissions. Centrally manages all technical/non-technical events, registration data, volunteer host assignments, and publishes final certificate placements.
Registrat	

Authentication Bootstrapping:

To prevent open registration for host accounts, the option to self-create administrative accounts is disabled. All host, registration, and admin credentials are pre-configured in firebase authentication. On login, the system automatically checks the user profile collection. If a matching profile is missing, it auto-bootstraps permissions according to the email mapping matrix.

4. System Architecture & Component Specifications

4.1. Secure Login & Bootstrapping Panel

- * Interface Branding: Main landing login portal proudly displays the title: "AltheronML 2k26" representing Kuppam Engineering College's technical edition.
- * Preset Account Flow: User registration on the login page is disabled to prevent credential exploits. Volunteers must log in with preset official email credentials.
- * Quick Pre-fill Bubble: To support fast testing, a floating shortcut dock displays preset credentials for admin, registration, and host accounts (e.g. registration@aitheronml.in, paperpresentation@aitheronml.in).

4.2. Public Self-Registration & Pass Issuance

- * Public Mode URL: Accessible via a dedicated bypass parameter (?mode=register&hideAdminSignIn=true), which opens a simple public entry form.
- * Team vs Individual Support: Accommodates both single entrants and team divisions. For team registrations, only the Team Name and Leader information are collected to minimize input friction.
- * HD QR Code Card: Upon successful registration, the participant receives a high-fidelity entry pass. Includes a vertical card layout with download-as-image capability and copy-link shortcuts.
- * Clipboard Copy Fallback: In cases where the page is loaded over non-secure HTTP local connections (where navigator.clipboard is blocked by the browser), the system triggers a fallback text-area command to guarantee copying.

4.3. Registration Desk Console (Volunteers)

- * Collapsible Layout: Features a sliding sidebar navigation layout that collapses completely (toggled via a menu icon). Collapsing frees up horizontal space, allowing the check-in roster and participant details panels to expand and fit side-by-side on tablet/desktop views.
- * Live Roster Table: Displays participant names, IDs (SYM-XXXXXX format), emails, phones, and registered event names. Includes instant status toggles for Payment Status (Paid / Pending) and Attendance Status (Present / Absent) that write directly to Firestore.

4.4. Event Host & Evaluation Console

- * Event Configuration: Enables hosts to set the event's physical venue/location, assigned coordinators, and time scheduling directly from the dashboard.
- * Batching System: Hosts can group checked-in participants into discrete batches (e.g. Batch 1, Batch 2) to manage evaluation phases. Displays the exact number of participants assigned per batch.
- * Criterion Setup: Flexible criteria editor where hosts configure evaluation parameters (e.g. Presentation, Technical Depth, Q&A) and set scoring weight weights (max points per criterion).
- * Scoring Ledger: Interactive tables for entering scores per participant. Integrates autosave capability to protect evaluation progress.
- * Placements and Publication: Automatically tallies scores, lists ranking recommendations (1st, 2nd, 3rd), and offers a single-click button to publish results to the public domain.

4.5. Live Support Helpline Integration

- * Custom Help Center Modal: Triggers an overlay modal when clicking the help icon in the Admin or Host Dashboard.
- * Support Hotline: Displays the coordinator's primary contact phone number: 8121280857.
- * Mobile Call Integration: Includes an anchor shortcut (`<a href="tel:8121280857">`) that triggers the native phone application dialer on mobile devices.

5. Database Architecture & Firestore Schemas

Google Firestore is utilized as the central database. The data model is structured around seven primary collections, optimized for fast document reads and real-time updates.

Collection: users

Document ID: email (string)

Fields:

- name (string): Display name
- role (string): Access permissions (superadmin / host / judge)

Collection: participants

Document ID: participantId (string - SYM-XXXXXX)

Fields:

- name (string): Student name
- college (string): Academic institution
- branch (string): Engineering branch
- email / phone (strings): Contact details
- eventId (string): Registered event ID
- attendanceStatus (string): Present / Absent / Pending
- paymentStatus (string): Paid / Pending
- createdAt (string): ISO timestamp

Collection: events

Document ID: eventId (string)

Fields:

- title (string): Event name
- category (string): Technical / Non-Technical
- venue (string): Room or seminar hall location

- status (string): Live / Upcoming / Completed
- registeredCount / checkedInCount (numbers): Attendance tracking
- hostEmail (string): Assigned host email

Collection: teams

Document ID: teamId (string)

Fields:

- teamName (string): Registered group name
- leaderId (string): Leader participant ID
- memberIds (array of strings): Member IDs
- eventId (string): Event reference

Collection: batches

Document ID: batchId (string)

Fields:

- name (string): Batch display title
- eventId (string): Event reference
- status (string): Waiting / Active / Completed
- participantIds (array of strings): Checked-in members assigned

Collection: results

Document ID: resultId (string)

Fields:

- eventId (string): Event reference
- rank1 / rank2 / rank3 (strings): Winner details
- scoreDetails (map): Scores aggregated
- published (boolean): Placement visibility flag

6. Technical Stack & Non-Functional Requirements

- * Frontend Stack: React 19 + TypeScript + Vite + Tailwind CSS.
- * Database Service: Google Firestore with real-time listeners (`onSnapshot`) to facilitate live synchronization.
- * Styling Directives: Standardize colors around premium material tokens:
 - Primary: Deep Indigo (#080C5F / #212773)
 - Surface / Backgrounds: Sleek M3 container tones with subtle outline-variant borders.
 - Glassmorphic Accents: Subtle blur-xs overlay wrappers on modal containers.
- * Security Rules: Firestore rules restrict writes to participant collections for unauthenticated users, permitting read/write access to score metrics only for hosts/judges.

7. Offline Synchronization Architecture

A critical feature of the system is the Offline Queue Sync layer which mitigates the risk of campus network dropouts:

1. Connection Listener: Monitors online/offline network events. When connectivity is lost, it flags the system as working offline.
2. Operations Queue: Administrative write operations (e.g. check-ins, registration updates, payments) are intercept-routed. Instead of crashing, the requests are assigned unique IDs and pushed to a persistent local cache queue (`localStorage`).
3. Live Queue Playback: Once the connection is restored, the synchronization manager replays queued operations sequentially, executing write operations to the Firestore server and ensuring zero loss of operational records.